Class Work: File Handling in Python

General Instructions for Students

Constraints

- Use **functions** to modularize the program.
- Use **file handling** (open(), read(), write(), append()).
- Use **lists/dictionaries** wherever appropriate.
- Do **not use databases** or external libraries.
- Ensure that updates are reflected in the original file.

Problem Statement 1: File Content Analyzer

A publishing company maintains articles in text files and wants to generate basic statistics about the content stored in these files. As a software developer, you have been assigned the task of creating a program that reads the contents of a text file and analyzes the information present in it.

Requirements

Write a Python program that reads data from a text file and displays the following details:

1. Total number of **vowels** present in the file.
2. Total number of **characters** present in the file (including spaces and special characters).
3. Total number of **lines** present in the file.

Sample File Content (article.txt)

Python is an easy-to-learn programming language.
It supports multiple programming paradigms.
File handling allows programs to work with data stored in files.

Expected Output

File Analysis Report

Total Number of Vowels   : 42
Total Number of Characters: 153
Total Number of Lines   : 3

Problem Statement 2: File Backup Utility

An organization wants to create backups of important text files to prevent accidental data loss. You have been asked to develop a utility that creates an exact copy of an existing file.

Requirements

Write a Python program that reads the entire contents of a source file and copies them into another destination file.

The program should:

1. Accept the names of the **source file** and **destination file** from the user.
2. Read the complete contents of the source file.
3. Write the contents into the destination file.
4. Display a success message after the copying process is completed.

Sample Source File (notes.txt)

Functions help in code reusability.
File handling enables persistent storage.
Python provides various modes to work with files.

Expected Output

Enter Source File Name   : notes.txt
Enter Destination File Name : backup.txt

File copied successfully.
All contents from 'notes.txt' have been copied to 'backup.txt'.